

TransitOps - Backend Schema Document

Database: Neon PostgreSQL | ORM: Drizzle ORM | Backend: Express.js + TypeScript

Main Database Tables

users

id(UUID PK), full_name(VARCHAR), email(VARCHAR UNIQUE), password_hash(TEXT), role_id(UUID FK), created_at(TIMESTAMP), updated_at(TIMESTAMP)

roles

id(UUID PK), name(VARCHAR UNIQUE), description(TEXT)

sessions

id(UUID PK), user_id(UUID FK), refresh_token(TEXT), expires_at(TIMESTAMP), created_at(TIMESTAMP), revoked(BOOLEAN)

vehicles

id(UUID PK), registration_no(VARCHAR UNIQUE), model(VARCHAR), type(VARCHAR), max_load_kg(INT), odometer(INT), acquisition_cost(DECIMAL), status(ENUM), created_at(TIMESTAMP)

drivers

id(UUID PK), name(VARCHAR), license_no(VARCHAR UNIQUE), category(VARCHAR), expiry_date(DATE), phone(VARCHAR), safety_score(INT), status(ENUM), created_at(TIMESTAMP)

trips

id(UUID PK), vehicle_id(UUID FK), driver_id(UUID FK), source(TEXT), destination(TEXT), cargo_weight(INT), planned_distance(INT), final_odometer(INT), fuel_used(DECIMAL), status(ENUM), created_by(UUID FK users), created_at(TIMESTAMP)

maintenance_logs

id(UUID PK), vehicle_id(UUID FK), title(VARCHAR), description(TEXT), cost(DECIMAL), status(ENUM), opened_at(TIMESTAMP), closed_at(TIMESTAMP)

fuel_logs

id(UUID PK), vehicle_id(UUID FK), liters(DECIMAL), cost(DECIMAL), date(DATE), created_by(UUID FK users)

expenses

id(UUID PK), vehicle_id(UUID FK), category(VARCHAR), amount(DECIMAL), description(TEXT), expense_date(DATE), created_by(UUID FK users)

Relationships

- roles 1 → many users
- users 1 → many sessions
- vehicles 1 → many trips
- drivers 1 → many trips
- vehicles 1 → many maintenance_logs
- vehicles 1 → many fuel_logs
- vehicles 1 → many expenses
- users 1 → many trips (created_by)
- users 1 → many fuel_logs (created_by)
- users 1 → many expenses (created_by)

Primary Keys

Every table uses a UUID primary key named id.

Foreign Keys

users.role_id → roles.id
sessions.user_id → users.id
trips.vehicle_id → vehicles.id
trips.driver_id → drivers.id
trips.created_by → users.id
maintenance_logs.vehicle_id → vehicles.id
fuel_logs.vehicle_id → vehicles.id
fuel_logs.created_by → users.id
expenses.vehicle_id → vehicles.id
expenses.created_by → users.id

Indexes

- users.email (UNIQUE)
- roles.name (UNIQUE)
- vehicles.registration_no (UNIQUE)
- drivers.license_no (UNIQUE)
- vehicles.status
- drivers.status
- trips.status
- trips.vehicle_id
- trips.driver_id
- maintenance_logs.vehicle_id
- fuel_logs.vehicle_id
- expenses.vehicle_id

Authentication & Session Handling

JWT Access Token + Refresh Token strategy.
Access token stored in HTTP-only cookie.
Refresh token persisted in sessions table.
Passwords hashed using bcrypt.
Role-Based Access Control enforced through middleware.

Permissions

Fleet Manager: Vehicles, Maintenance, Dashboard.
Dispatcher: Trips and Driver Assignment.
Safety Officer: Driver Management & Compliance.
Financial Analyst: Fuel, Expenses, Analytics.
Unauthorized users cannot access protected APIs.

Data Ownership Rules

- Vehicles own Trips, Fuel Logs, Maintenance Logs and Expenses.
- Users own Sessions and created records.
- Deleting a vehicle is restricted if linked records exist.
- Business rules prevent assigning unavailable vehicles or drivers.
- Registration numbers and license numbers must remain unique.